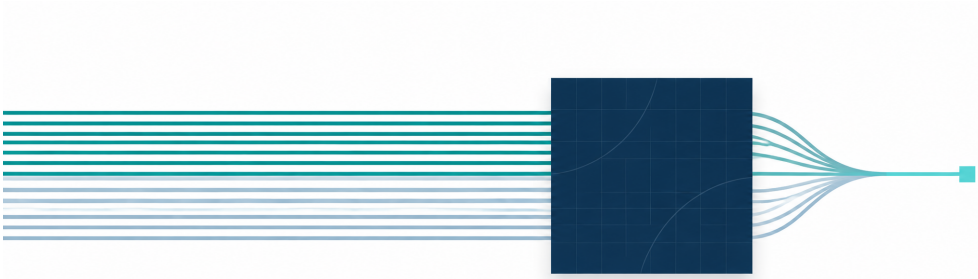
香山昆明湖 V2

自定义向量点积指令

第一阶段执行日志与过程文档

技术报告



工程组合： 0f ＋ firstdot V2 ＋原 C8

团队名称： 待团队确认

成员： 待团队确认

学校： 待团队确认

材料日期： 2026 年 11 月 2 日

文档修订： 2026 年 11 月 5 日

目录

第一阶段：环境验证与向量执行过程 1

1.1 验证环境与协同仿真框架 1

1.1.1 构建链路与运行产物 1

1.2 环境部署与运行步骤 3

1.2.1 WSL 资源配置 3

1.2.2 基础工具安装 4

1.2.3 JDK 安装与校验 4

1.2.4 Verilator 安装与环境核查 4

1.2.5 程序构建环境 5

1.2.6 集成构建与运行入口 5

1.2.7 构建资源与并行恢复 8

1.2.8 程序镜像、装载与启动 9

1.3 启动差分故障的定位与修复 10

1.4 hello 与 vadd 运行结果 12

1.4.1 10 月 25 日英文输出与运行验证 13

1.4.2 定宽加法与字节结果 13

1.5 基于波形的向量执行过程 14

1.5.1 执行、写回与提交 15

1.5.2 指令、事件与波形联合定位 17

1.5.3 10 月 25 日含时钟的波形调试记录 18

1.6 阶段结论与复现资料 18

1.6.1 阶段结论与材料入口 18

1.6.2 源码版本与依赖恢复 18

1.6.3 完整工具版本 19

1.6.4 跨环境复现条件 20

1.6.5 10 月 25 日验证的版本与产物 21

1.6.6 技术资料索引 21

1.7 原始操作日志 **21**

1.7.1 10 月 25 日生产模型原始输出 21

1.7.2 10 月 25 日 FST 模型原始输出 22

1.7.3 程序构建原始日志 22

1.7.4 历史 hello 与 vadd 运行摘要 23

1.7.5 NEMU 构建原始日志 24

1.7.6 EMU 构建原始日志 26

1.7.7 工具与编译链版本原始输出 28

1.7.8 固定子模块版本清单 29

1.7.9 原始波形事件记录 32

1.7.10 向量运算单元事件记录 33

第一阶段：环境验证与向量执行过程

工程组合：0f ＋ firstdot V2 ＋原 C8　材料日期：2026 年 11 月 2 日

本阶段记录香山仿真环境部署、hello/vadd 运行与向量执行调试。2026 年 10 月 9 日记录展示环境建立、启动差分修复和原波形过程；10 月 25 日的部署复核使用既有硬件构建产物，运行配置个人标识的英文 hello 程序，并完成含时钟信号的 FST 调试链。两组实验分别按各自源码、输入镜像与事件坐标说明。

1.1　验证环境与协同仿真框架

本阶段建立香山 RTL 仿真器、NEMU 参考模型与裸机程序协同工作的运行入口。hello 程序用于检查启动流程和串口输出，vadd 程序用于观察向量配置、数据加载、按元素运算、写回与提交。通过共同启动镜像和差分比较，将工具链生成的产物与实际运行结果连接起来，为后续指令扩展提供可复用的环境。

1.1.1　构建链路与运行产物

协同仿真由硬件执行端、软件参考端和目标程序共同组成。香山 EMU 负责模拟硬件的执行过程，NEMU reference 负责按指令语义推进参考状态，裸机程序提供可重复的输入并检查软件可见结果。共同 flash 连接复位后的启动过程与程序入口。这几类产物在一次运行中同时参与工作，部署过程因此围绕它们之间的接口组织。

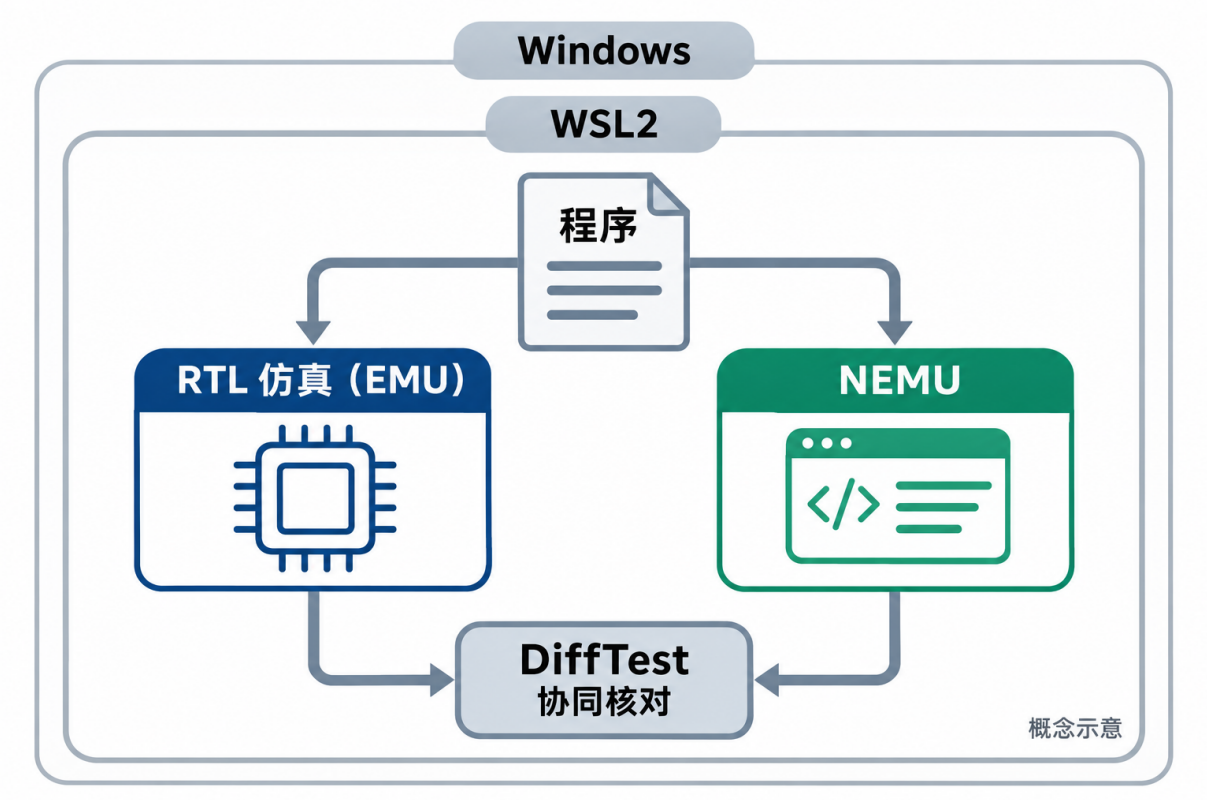
硬件侧首先通过 Java 与 Mill 解析 Scala、Chisel 工程及其依赖，再由 firtool 处理硬件中间表示，供 Verilator 生成宿主侧仿真代码。宿主 C++ 编译器将这些代码构建为 EMU。每一步的产物都会成为下一步的输入：Scala 工程配置影响生成的硬件，硬件生成结果影响仿真代码，宿主工具链则决定最终可执行文件的构建方式。运行环境的版本表对应这条生成链，便于沿产物关系定位问题。

目标程序采用另一条工具链。RISC-V GCC 编译程序与所需运行支持，链接器确定代码和数据布局，binutils 提供二进制转换及反汇编。NEMU reference 在宿主侧构建为可加载的参考库，由 EMU 运行参数指定。这样，硬件仿真器、目标程序和参考模型具有各自清楚的构建入口，并在差分运行时汇合。

实际组织时，构建日志与运行日志分别保存。构建记录说明工具如何生成产物；运行记录说明加载器选择了哪些文件、程序如何启动以及最后如何结束。两者通过产物路径、文件摘要和运行参数对应。对一个具体问题，先找到相关产物，再沿依赖链向上查看生成步骤，能够较快区分源码、工具和装载配置三个层面的原因。

表 1-1 构建链路与运行产物

产物	生成入口	运行中的作用	主要定位信息
香山 EMU	硬件生成与宿主编译	执行 DUT 仿真	硬件版本、配置、构建参数、文件摘要
NEMU reference	参考模型构建	提供架构状态参考	源码版本、配置、加载路径、文件摘要
程序 ELF/BIN	RISC-V 编译与链接	执行 hello 及 vadd 测试	入口、装载段、反汇编、镜像摘要
共同 flash	启动汇编与二进制转换	建立一致的启动输入	文件长度、内容摘要、显式加载参数



概念示意：共同输入与软硬件状态核对。技术依据见图 1-1。

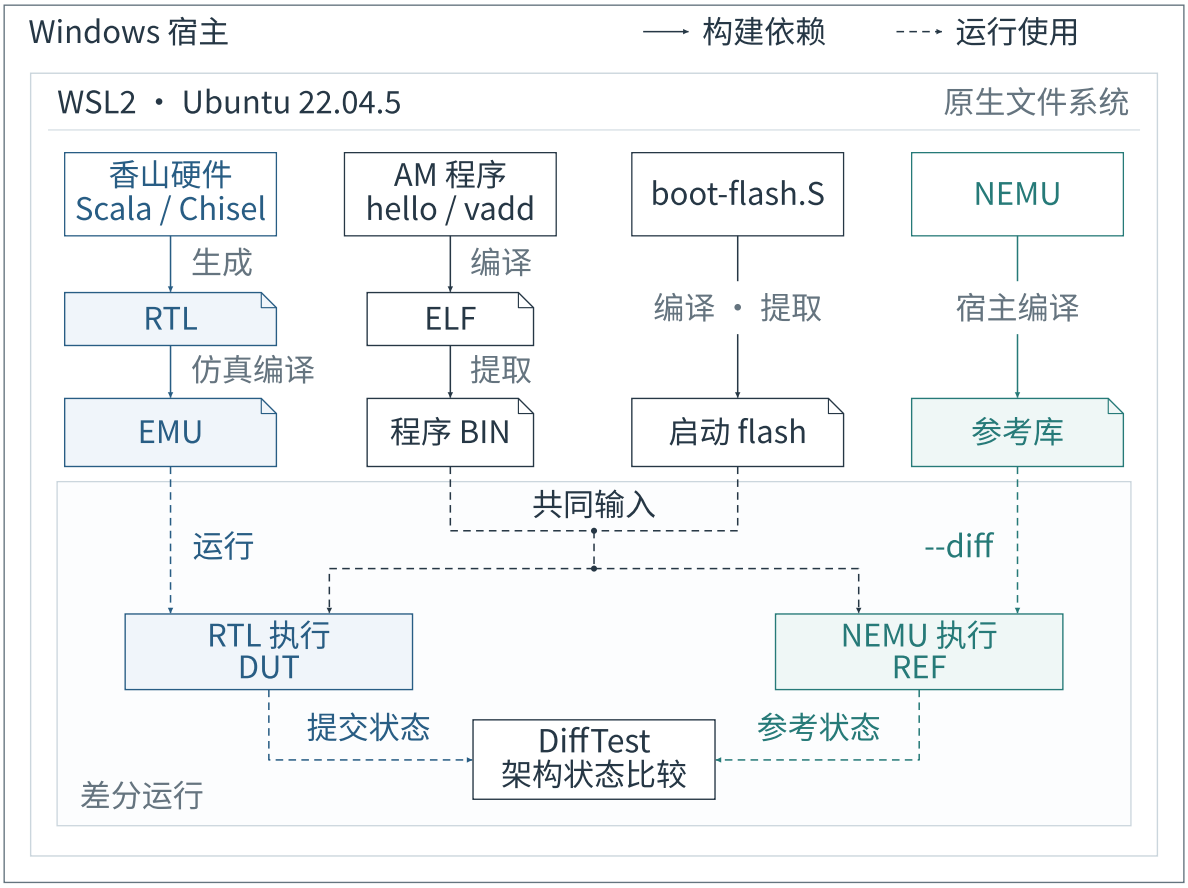


图 1-1 开发环境与软硬件协同验证框架

Windows 宿主中的 WSL2 环境承载构建与差分运行。实线为构建依赖，虚线为运行使用；程序 BIN 与启动 flash 是共同输入，EMU 和 NEMU 参考库分别提供 DUT 与 REF 功能。框图不表示独立操作系统进程或内部逐周期锁步。阶段一历史环境，未混入最终 0f 身份。

1.2 环境部署与运行步骤

1.2.1 WSL 资源配置

本次为 WSL 分配 20 GB 内存和 16 GB 交换空间。Windows 用户目录%UserProfile%下的.wslconfig 内容如下：

```
[wsl2]
memory=20GB
swap=16GB
```

保存正在进行的 WSL 工作后，在 PowerShell 执行 wsl --shutdown，重新进入 Ubuntu，再用 free -h 核对生效结果。原机器的交换文件路径为 D:\WSL\wsl-swap.vhdx；新环境按本机磁盘布局选择路径即可。

内存规划需要同时考虑 JVM、硬件生成工具和并行 C++ 编译进程。交换空间可为短时内存压力提供缓冲，但频繁换页会增加构建时间，因此它与可用物理内存具有不同作用。内存较小的机器应相应降低 JVM 堆与编译并行度，使多个进程的合计需求处于

可用范围内。构建日志同时记录并行参数和结束状态，便于区分资源配置问题与源码编译问题。

1.2.2 基础工具安装

完成配套源码检出后，在团队仓库根目录加载路径，并安装基础工具：

```
source team/scripts/activate.sh
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target base
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target llvm
sudo bash "$TEAM_XS_ENV_ROOT/setup-tools.sh" --target mill
sudo apt-get install -y cmake ninja-build gtkwave
```

入口来自固定 xs-env 的 setup-tools.sh。软件源提供的补丁版本可能随时间变化，安装步骤完成后仍需通过环境检查确认实际版本。这样可以把“安装入口相同”与“得到的工具版本相同”分别记录。

1.2.3 JDK 安装与校验

原安装入口使用 Oracle 21/latest 地址。为了恢复本阶段的 21.0.12，团队固定 GraalVM 安装脚本使用对应 Linux x64 归档及官方 SHA-256，安装到用户目录：

```
export JAVA_HOME="$HOME/.local/share/graalvm-jdk-21.0.12"
bash team/scripts/install-graalvm.sh
source team/scripts/activate.sh
```

归档校验用于确认下载字节，版本检查用于确认实际启动的 Java。二者配合，可以发现下载路径正确但环境变量仍指向另一套 JDK 的情况。新终端继续导出相同 JAVA_HOME。原环境中已有 /opt/graalvm-jdk-21 的安装，也可以在确认版本后沿用；脚本遇到版本不同的已有安装会停止，以保留当前配置。

安装方法参考 Oracle GraalVM 安装说明。固定下载项为 21.0.12 Linux x64，校验来源为官方 SHA-256。工具更新时同步记录新版本及相应验证结果，便于将新产物与本阶段历史产物区分。

1.2.4 Verilator 安装与环境核查

Verilator 从固定提交构建，以下流程显式使用两个编译任务。构建并行度应与本机可用资源相配合；阶段一 EMU 编译记录中的四任务参数与这里安装工具本身的参数分别记录。

```
sudo apt-get install -y help2man perl autoconf flex bison libgoogle-perftools-dev \
  libjemalloc-dev numactl perl-doc zlib1g-dev liblz4-dev libfl2 libfl-dev
mkdir -p "$HOME/riscv-tools"
git clone https://github.com/verilator/verilator.git "$HOME/riscv-tools/verilator"
```

```
cd "$HOME/riscv-tools/verilator"
git checkout ea338be98e1e838d3518809ce8899f85a009963c
unset VERILATOR_ROOT
autoconf
./configure CC=/usr/local/llvm/bin/clang CXX=/usr/local/llvm/bin/clang++ LINK=/usr/local/llvm/bin/clang++
make -j2
sudo make install
```

编译前清除旧的 VERILATOR_ROOT，并显式选择已安装的 Clang，以便构建输入与记录一致。安装后回到团队仓库根目录检查环境：

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
```

环境检查核对配套源码提交、香山递归子模块和关键工具版本。香山提交可以包含本队新增文档，但应保留指定硬件基线祖先。检查结果用于确认构建条件；硬件行为则由对应版本的仿真记录说明。这样的分工使环境恢复与功能验证各自具有清楚的输出，也便于定位失败发生在哪一层。

1.2.5 程序构建环境

原记录的工作目录为 /home/riscv/riscv-lab/xs-env/XiangShan。复用其他机器或其他检出时，在实际团队仓库根目录加载环境即可，并保留本次路径和提交记录。

activate.sh 将当前团队仓库设置为 NOOP_HOME，按照 TEAM_XS_ENV_ROOT 指向配套 NEMU 和 AM，并设置 LINUX_GNU_TOOLCHAIN=1。这些变量把硬件仓库、参考模型和程序构建入口连接起来，运行前可据此核对各组件来自哪套环境。

AM 构建向量工作负载时仍需显式指定 MARCH=rv64gcv。这使编译器采用包含向量扩展的目标 ISA，与 hello-vadd 所用的向量装载、加法和存储指令一致。程序构建完成后，将 ELF、运行镜像、反汇编和实际运行命令一起归档，便于从源程序追踪到日志中的提交 PC。

1.2.6 集成构建与运行入口

原实验使用 Windows 下的 WSL2 Ubuntu 22.04.5，宿主处理器为 Ryzen 7 7840H，可用物理内存约 27.7GiB。WSL 配置 20GB 内存与 16GB swap，硬件生成、编译和仿真在 WSL 原生文件系统中执行。下表列出阶段一的主要版本与运行设置；完整子模块锁和工具输出列于文后。

表 1-2 集成构建与运行入口

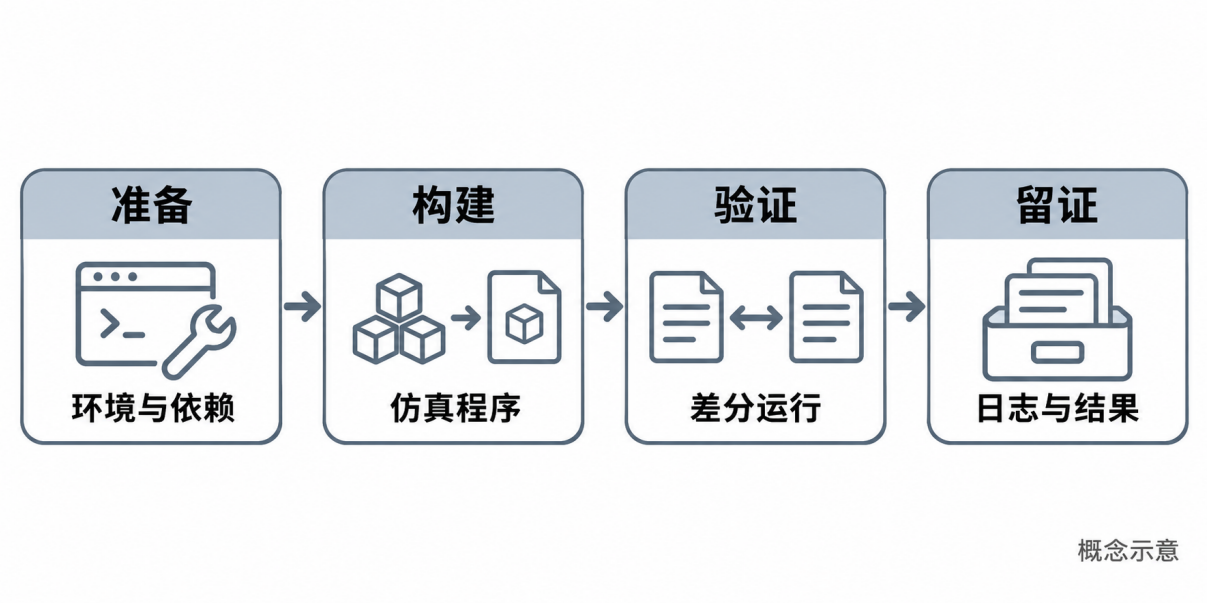
组件	固定版本或设置
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
原香山基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
原 NEMU 基线	a48a11f208ff81ad765591d83271d1b339fb3861
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
JDK / Mill / firtool	21.0.12 / 0.12.3 / 1.62.1
Verilator	5.052，固定 ea338be 提交
RISC-V GCC / binutils	11.4.0 / 2.38
DUT	MinimalConfig、单核、VLEN128
原波形构建	EMU_THREADS=1、EMU_TRACE=fst

部署按照依赖关系推进：配置 WSL 资源，检出配套源码和递归依赖，核对工具版本，分别构建 NEMU reference 与香山 EMU，编译 hello/vadd 及启动镜像，最后执行差分运行。命令、退出状态、版本和文件摘要随各阶段产物一起保存。

复现入口由以下脚本组成。工作负载中的团队标识由成员确认后填写；构建与运行输出应使用独立目录，以便对应具体实验。

```
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID= 正式团队标识 bash team/scripts/stage1.sh workload
bash team/scripts/stage1.sh run
```

部署中遇到的首个差分问题位于启动 PC 0x10000004。检查发现两端默认 flash 分别包含 3 条和 6 条启动指令。随后使用同一 boot-flash.S 生成 12 字节镜像，并在启动命令中显式指定共同 flash、128MB RAM 和固定 reference。完整定位过程见后文“启动差分故障的定位与修复”。



概念示意：部署、运行与证据归档。技术依据见图 1-2。

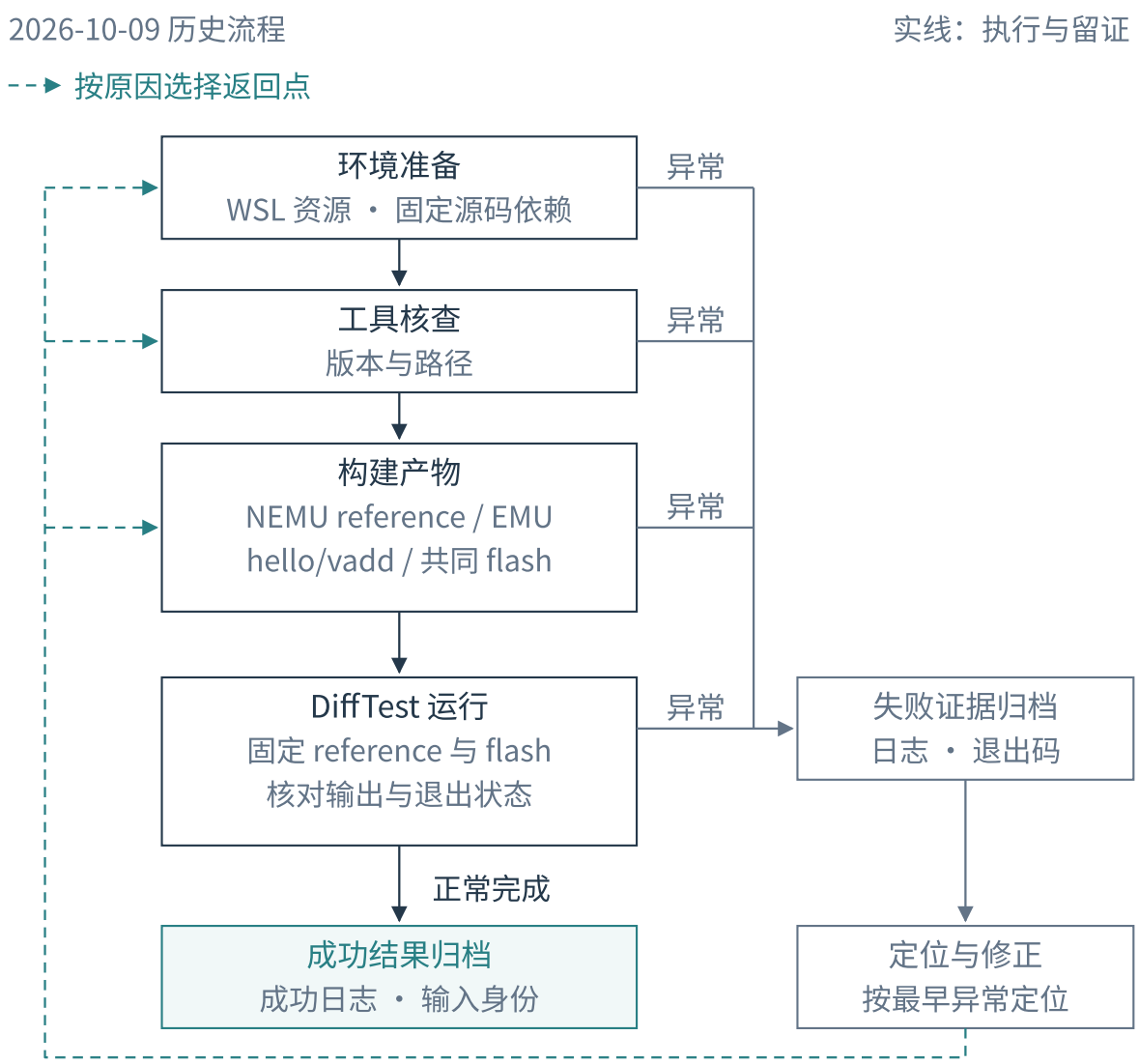


图 1-2 环境部署与验证执行流程

环境准备、工具核查、产物构建、DiffTest 运行和成功归档构成主流程；失败证据另行归档。定位修正后按故障原因返回准备、工具核查或构建环节，虚线为处理路径，不表示每一类历史故障均实际发生，也不表示脚本自动诊断或修复。2026-10-09 阶段一流程：香山 16ae9dd、NEMU a48a11f。

1.2.7 构建资源与并行恢复

香山 EMU 的构建涉及多个资源特征不同的阶段。Java 进程需要堆空间解析工程和生成硬件，firtool 处理硬件表示，Verilator 生成仿真代码，C++ 阶段则启动多个编译进程并产生对象文件。宿主文件缓存与操作系统也占用内存。资源配置要为这些部分保留共同工作的空间，单独增大某一进程的内存上限并不等于增加了整条构建链的可用资源。

原实验主机可用物理内存约 27.7GiB，WSL 配置 20GB 内存和 16GB swap。这里的 20GB 是 WSL 内存上限，swap 用于承接部分内存压力；二者在访问成本和作用上不同。持续发生换页时，构建墙钟时间可能明显增加。因此，资源记录同时关注内存配置、并行度、进程状态和日志进展，而不是只记录一项内存容量。

2026 年 10 月 9 日的构建记录中，C++ 编译在 20:42:15 中断并恢复，并行度调整为

4，已完成对象保留；EMU 随后在 20:53:32 完成构建。这个过程体现了分阶段调节并发的方式。复现脚本中的较低默认并发适合控制初次构建的内存压力，原记录中的-j4 则对应当时的实际执行参数。两种配置都通过各自命令和时间记录定位。

并行资源也应与仿真配置分开描述。阶段一 DUT 采用单核，原波形构建使用单仿真线程；宿主可以同时运行多少独立任务，属于另一项调度条件。构建并发影响宿主资源和等待时间，程序的 EMU 周期统计则来自仿真内部。将构建耗时、宿主执行时间和 DUT 周期分别记账，便于后续评价环境效率与指令性能。

对象文件和构建缓存有助于恢复长作业，但它们对应具体源码、工具和编译参数。沿用已完成对象时，继续保存原命令与依赖身份；调整会影响生成内容的参数时，按照构建系统的依赖关系更新相应产物。报告保留中断和恢复记录，使最终 EMU 的形成过程具有连续的解释，而非只有最后一条成功提示。

- 2026 年 10 月 9 日 20:42:15，中断并恢复 C++ 构建，将并行度调为 4，保留已完成对象。
- 2026 年 10 月 9 日 20:53:32，EMU 构建结束，状态文件记录 exit_code=0。
- 2026 年 10 月 9 日 20:55:26，workload 与共同 flash 构建结束，记录 WORKLOAD_BUILD_PASS。
- 原正常运行于 21:00:28 结束，退出状态为 0。

1.2.8 程序镜像、装载与启动

ELF、BIN 和 flash 在运行中承担不同作用。ELF 包含符号、代码与数据段信息，可用于查看入口地址和反汇编；BIN 提供加载器使用的程序字节；flash 提供复位后的启动代码。启动镜像与目标程序是相互配合的两个输入，分析程序尚未到达 C 入口的差异时，flash 及装载配置通常是重要线索。

反汇编连接源程序和硬件观察点。编译器选择指令，链接器安排代码与数据，最终地址由这组构建条件共同决定。通过目标指令的 PC 查找同次构建的机器字，再与提交记录对应，可以确定波形中观察的是哪一次操作。即使 C 代码保持原样，链接参数或依赖库改变后，PC 表也可能随之变化。

显式运行参数把构建产物与一次执行绑定。阶段一在命令中指定共同 flash、128MB RAM 与固定 reference，使两端启动输入可以直接检查。共同文件的含义是字节实物一致；两端如何将它装入各自平台地址空间，仍由各自装载实现决定。描述共同启动条件时，文件内容、装载参数和地址映射分别列出，因果关系更清楚。

hello 程序提供了启动链路中的一个软件观察点。串口输出表明程序已进入对应的软件路径，随后 vadd 用例设置向量状态、执行运算并打印比较结果。原 hello 中的 IPv4 来自 WSL 宿主，用于识别当时环境；团队字符串为原记录中的 riscv-vdot-team。正式身份信息与历史输出分别保存，便于在更新展示材料时继续检索原运行。

装载问题与运算问题也具有不同的日志特征。启动日志给出 flash、程序和 reference 的选择，软件输出给出已到达的程序阶段，差分信息给出首次观察到的架构差异。按执行顺序阅读这些信息，可以先确定故障发生在启动之前、启动过程中还是进入测试程序之后，再选择相应源码和数据继续分析。

1.3 启动差分故障的定位与修复

原失败的首次差异位于 PC 0x10000004，涉及 t0 寄存器，记录的 instrCnt 为 2。这个位置处于启动阶段，早于 hello/vadd 测试主体。定位首先围绕两端的初始执行序列展开：检查启动输入，查看该位置的机器字，并与启动文件的反汇编对应。

检查发现，两端默认 flash 分别包含 3 条和 6 条启动指令。默认输入差异使两端在到达同一程序入口前就经历了不同的执行过程。修复采用一份 boot-flash.S 生成共同的 12 字节 boot-flash.bin，再通过显式 flash 参数将同一实物交给 DUT 与 NEMU。三条启动指令的文件偏移、机器字与反汇编保留在图数据附件中。

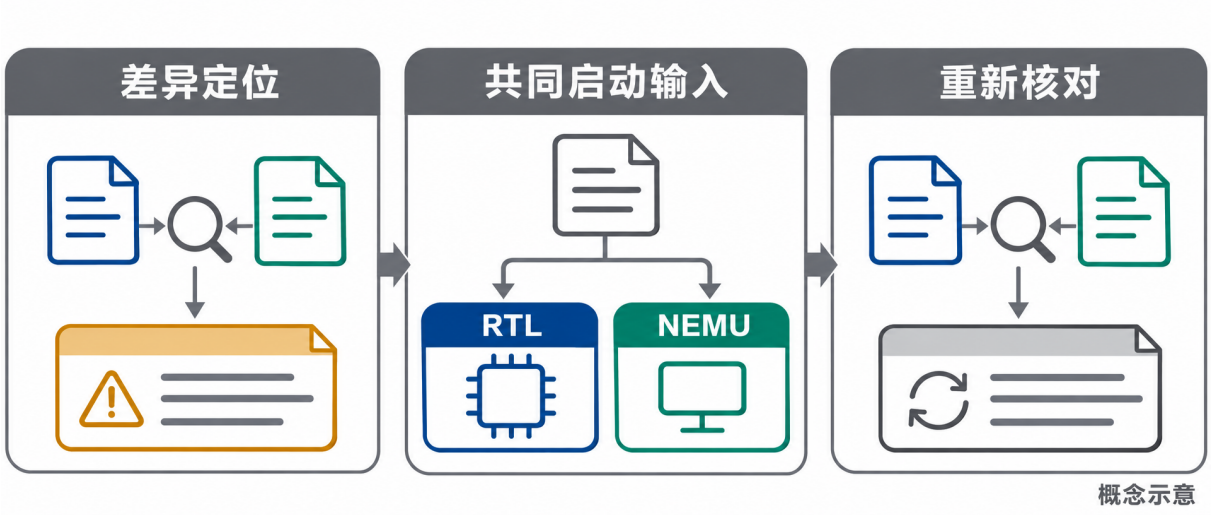
文件偏移定位 flash 中的指令，启动 PC 定位处理器执行地址，两者通过装载映射关联。同一份 12 字节 boot-flash.bin 分别交给 DUT 与 NEMU 装载。首次差异 PC 用于定位原故障，后续终止或 ABORT 事件的地址按对应事件记录。

恢复运行继续启用 DiffTest，执行三类 vadd 用例，记录 24 个元素通过、GOOD TRAP 和 exit 0。失败记录与恢复记录分别保存，前者解释问题如何被发现，后者说明统一启动输入之后的结果。报告中的修复过程由“首次差异—默认输入检查—共同文件—正常运行”组成，每一步都可以回到原日志或构建实物。

表 1-3 启动差分故障的定位与修复

排查步骤	使用材料	定位结果
查找首次差异	原失败日志、PC 与 t0 状态	差异发生于启动阶段
比较启动输入	默认 flash 与反汇编	两端默认指令序列不同
统一启动文件	同一 boot-flash.bin 及显式参数	两端加载同一 12 字节实物
查看恢复结果	软件摘要、GOOD TRAP 与退出状态	hello/vadd 完成正常差分运行

这一案例也给后续故障分析提供了合适的起点：先定位最早出现差异的阶段，再分析对应输入和执行状态。若差异仍在启动入口，应优先检查 flash 与装载；若已经进入向量程序，则转向指令、寄存器和结果记录。分层定位可以缩小检查范围，并使每次参数调整与观察结果直接对应。



概念示意：统一启动输入与差分恢复。技术依据见图 1-3。

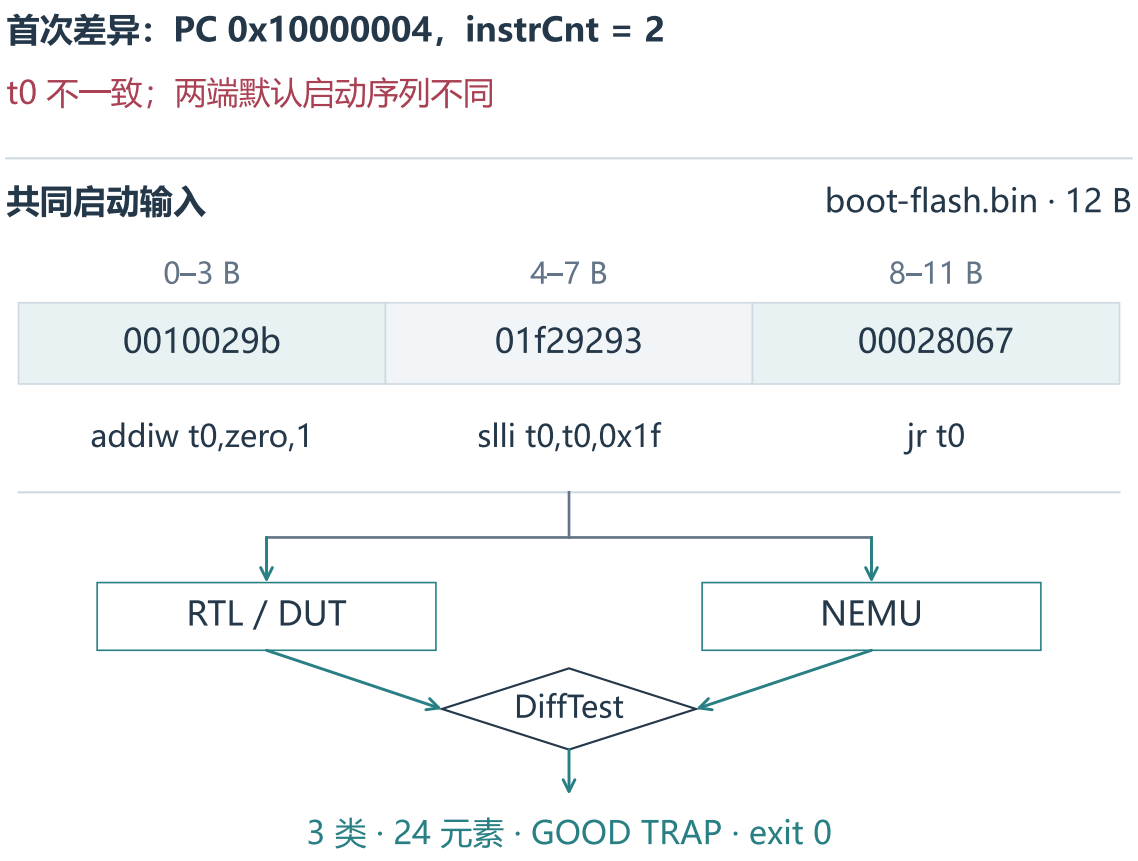


图 1-3　启动输入统一与差分恢复

首次差异位于启动 PC 0x10000004，instrCnt 为 2。共同 boot-flash.bin 的 12 字节由三条机器指令组成，图中 0–3、4–7、8–11 为文件字节偏移。显式向 RTL 与 NEMU 提供同一启动文件后，三类共 24 个元素通过检查，以 GOOD TRAP 和 exit 0 结束；共享文件不等于两端物理映射地址相同。

1.4 hello 与 vadd 运行结果

hello 输出包含团队标识和 WSL 宿主 IPv4。2026 年 10 月 9 日记录使用的团队字符串为 riscv-vdot-team，本文按原日志保留。正式团队名称等身份信息集中列于封面。

vadd 程序采用 SEW8、LMUL1 和 VL8，依次执行两次向量加载、一次 vadd.vv 及一次向量存储，再将八个结果字节与标量参考逐项比较。三个用例分别检查基本加法、负数补码和定宽回绕。

表 1-4 hello 与 vadd 运行结果

用例	输出字节
正数	9 9 9 9 9 9 9 9
负数补码	254 252 250 248 246 244 242 240
边界回绕	128 127 0 0 0 0 254 254

正常运行记录包含 HELLO_VADD_PASS cases=3 lanes=24、GOOD TRAP 和真实退出状态 0，运行期间启用 DiffTest。EMU 统计为 16557 条指令、42962 周期，统计对象是包含启动、UART 与结果校验的完整程序。各用例的数值解释见本节“定宽加法与字节结果”，事件关系见“基于波形的向量执行过程”。

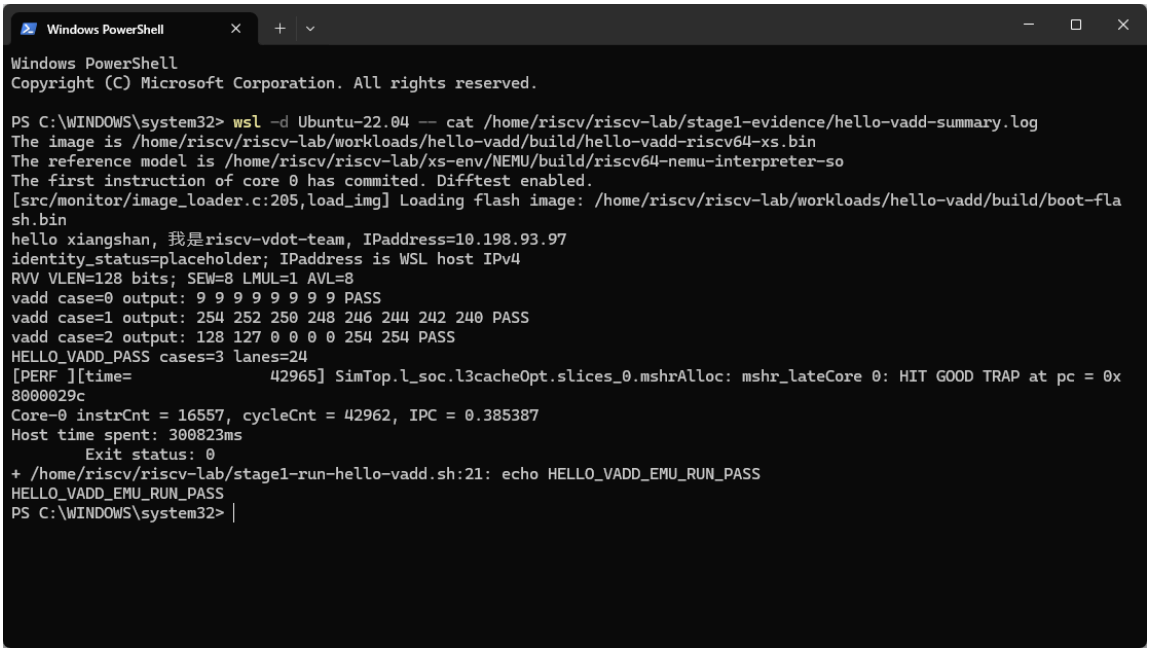


图 1-4 hello 与 vadd 正常结束输出

运行日志记录三类 vadd 输出、DiffTest 比较和 GOOD TRAP 终态。

1.4.1 10 月 25 日英文输出与运行验证

2026 年 10 月 25 日（UTC+8），补证实验使用个人标识 LiZufan001 构建 hello/vadd 程序，身份状态为 configured。原始串口输出如下，其中 IPv4 为构建宿主地址：

```
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
```

同一 BIN 分别在已封存生产 EMU 和独立 FST 模型上启用普通 NEMU DiffTest，两次运行均通过 3 组、24 个元素检查，记录HELLO_VADD_PASS cases=3 lanes=24、GOOD TRAP 及退出状态 0。每次统计均为 16621 条指令、43249 周期，覆盖启动、UART 和结果校验的整程序。原始输出与运行回执保存了具体输入、命令和结束状态。

1.4.2 定宽加法与字节结果

阶段一 vadd 采用 SEW8、LMUL1 和 VL8，参与用例比较的是八个 8 位元素。程序通过两次向量加载取得输入，执行按元素加法，再将结果存回内存供标量比较。三个用例使用相同的执行入口，分别选择便于观察基本运算、补码和回绕行为的数据，从而让环境验证具有不同的数值特征。

RISC-V V1.0 规范定义同宽向量整数加法，并说明未特别指定的整数运算在溢出时回绕。八位加法因此按模 256 保存结果。若把一个结果字节记为 r，它的无符号表示位于 0 到 255 之间；按八位补码解释时，最高位为 1 的字节对应负值。例如原负数用例打印的 254 是十六进制 FE，对应补码整数 −2。打印形式由格式选择决定，存储的八位比特保持相同。比较时以相同宽度和截断规则计算标量参考，就能直接核对程序结果。

正数用例的八个结果字节均为 9，低 64 位显示为 0x0909090909090909。负数用例按元素顺序打印 254、252、250、248、246、244、242、240，对应低 64 位 0xf0f2f4f6f8fafcfe。观察十六进制向量值时，右端是最低字节，因而与按元素序号递增打印的字节列表方向不同。把两个显示形式按字节位置对应，可以避免把书写顺序差异当成数据差异。

边界用例输出 128、127、0、0、0、0、254、254，低 64 位为 0xfefe000000007f80。这里同时出现最高位变化、零和接近字节上限的值，适合检查定宽结果的表示。八个字节在向量低 64 位中的组合关系可以写成各结果字节乘以相应的 256 次幂后求和；因此逐元素列表与整体十六进制值是同一结果的两种表达。

VLEN128 说明向量寄存器的物理宽度，VL8 说明该用例的活动元素数。本文结果表按实际八个元素解释，整体寄存器宽度与有效运算范围分别标注。随后 vdot 使用低八字节参与点积并形成自定义结果布局，属于另一套指令语义，其低 32 位结果和高位规则在阶段二说明。

软件输出还保留了用例数量与比较数量。HELLO_VADD_PASS 中的 cases=3 对应三种输入组织，lanes=24 对应三次各八个活动元素。读者可以将该摘要与逐字节结果及 GOOD TRAP、exit 0 一起阅读，得到从数值比较到运行结束的完整解释。

1.5 基于波形的向量执行过程

取指和译码之后，向量指令经重命名、调度与操作数准备进入执行单元。两个 vle8.v 提供输入，vadd.vv 完成按元素加法，结果进入写回路径并在 ROB 提交。差分系统在架构状态比较点与 NEMU 对应状态核对，内部观察器则记录输入、输出和提交事件。

原波形提取的 vadd 提交 PC 为 0x80000138，机器码为 0x021101d7，vecwen=1、wdest=3。下表按这组指令身份关联三个用例的事件坐标与低 64 位结果。

表 1-5 基于波形的向量执行过程

用例	输入事件坐标	输出事件坐标	提交事件坐标	运算输出低 64 位
正数	19625	19626	19631	0x0909090909090909
负数	25460	25461	25466	0xf0f2f4f6f8fafcfe
边界	33009	33010	33015	0xfefe000000007f80

三个观察点展示了执行结果产生到架构提交的流水关系。表中数字是原 CSV 事件坐标，图中间距用于组织阅读；现有局部截图未显示时钟，因而本节按事件位置解释记录，不把坐标差换算为物理时间或固定指令时延。需要进一步查看握手时，可恢复对应完整波形，联合定位时钟、valid、ready 和结果信号。

原归档记载的完整 FST 路径为/home/riscv/riscv-lab/stage1-evidence/hello-vadd.fst，文件大小 220295598 字节。该记录属于 2026 年 10 月 9 日基线，恢复时以归档索引和文件摘要定位。最终 0f 环境的 hello/vadd 验证记录由 R5 验收单独绑定。



概念示意：向量操作的执行与状态观察。技术依据见图 1-5。



图 1-5 vadd 输入、输出与提交事件

三类 vadd 用例通过 PC 0x80000138、指令 0x021101D7 和目的 v3 关联。数字为 CSV 事件坐标。

1.5.1 执行、写回与提交

程序中的一条向量指令会经过多个硬件阶段。取指和译码确定指令及操作类型，重命名与调度安排数据依赖，执行单元计算结果，写回路径传递结果与身份，ROB 按架构顺序提交。执行结果出现与架构提交是不同观察点，二者之间还存在父级选择及在途状态的处理。

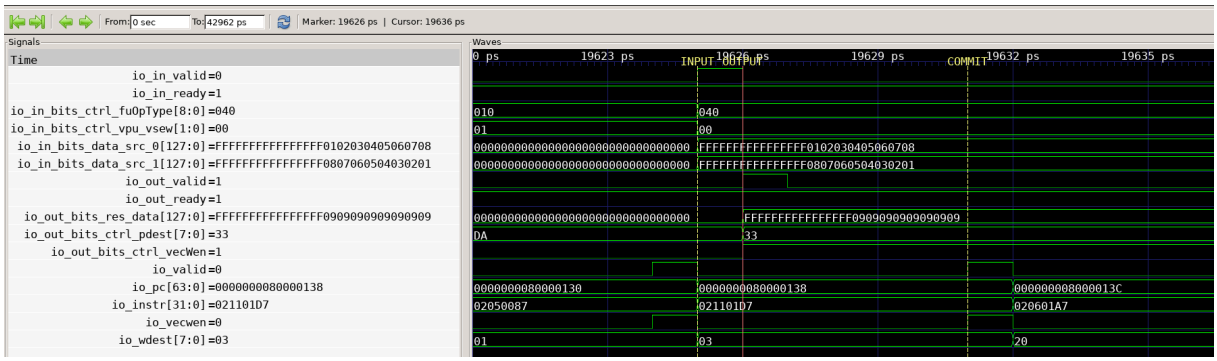
阶段一按 PC 0x80000138、机器字 0x021101d7 及目的 v3 关联 vadd 记录。提交侧 vecwen=1 说明该记录带有向量写入，wdest=3 给出对应目的。PC 和机器字共同提供静态指令位置，目的信息帮助确认被观察的结果。它们与用例顺序、事件坐标和结果值结合后，才能把三类软件输入准确连接到各自的硬件事件。

正数用例的输入、输出和提交事件分别位于 19625、19626 和 19631，负数用例为 25460、25461 和 25466，边界用例为 33009、33010 和 33015。这些数字沿用原 CSV 坐标，表示所列记录在波形中的位置。图中采用三列布局突出阶段关系，列间距离没有按原坐标比例绘制。

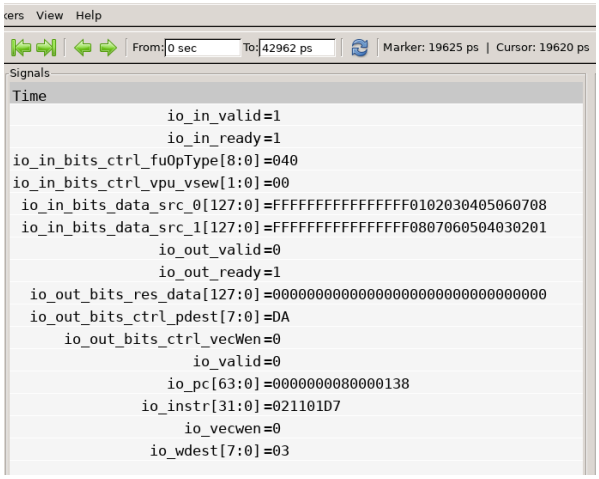
内部信号的含义还取决于采样位置。输入记录可能表示执行端可见的操作数，输出记录表示结果观察点，握手则需要结合 valid 和 ready 判断一次事务是否实际转移。阶段一图表按现有输入、输出与提交字段命名；需要讨论握手行为时，应在原波形对应位置同时查看双方条件。这样既能利用已有记录解释流水关系，也能清楚说明各字段承担的作用。

现有局部波形截图未显示时钟。截图中的 ps 是波形显示单位，事件坐标是用于定位的数值；EMU 周期统计和宿主运行时间则来自不同的计量位置。本阶段通过 PC、字段与坐标连接记录，没有为这些局部窗口建立物理频率或固定时延结论。

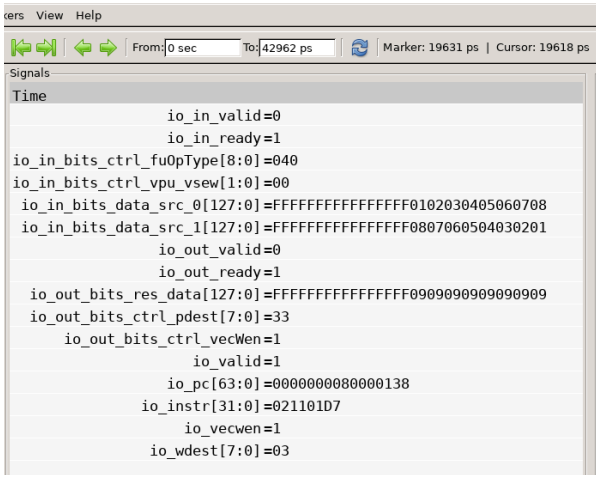
乱序执行中，程序依赖、执行资源及下游状态会影响执行和提交的间隔。报告用三个具体窗口说明该次运行的过程，后续性能分析则采用专门的 DUT 计数窗口和相同工作量对照。把事件定位与性能计量分开，读者就能分别回答“结果在哪里出现”和“规定工作用了多少周期”。



(a) 输出位置：19626 ps



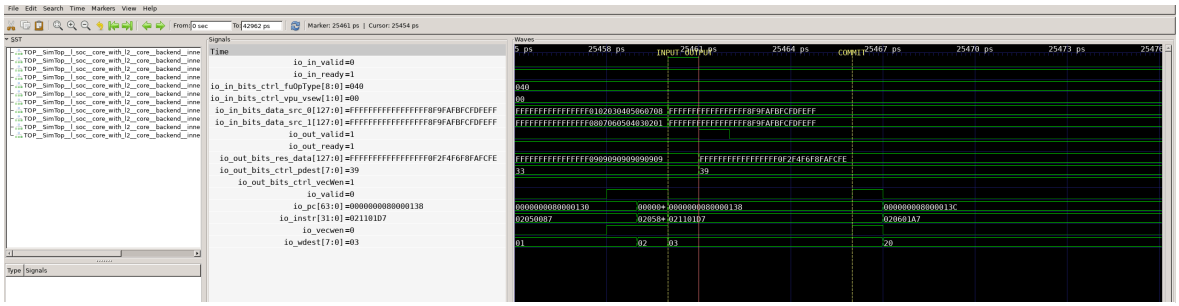
(b) 输入位置：19625 ps



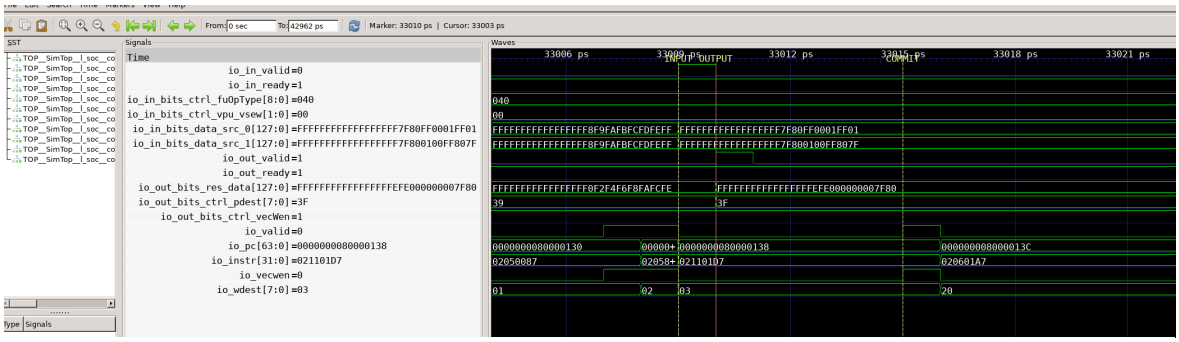
(c) 提交位置：19631 ps

图 1-6 vadd 正数用例的执行与提交波形

(a) 输出位置及相关事件；(b) 输入游标处信号；(c) 提交游标处信号。19625、19626 和 19631 为 selected VCD 中的事件坐标，截图单位为 ps，时钟信号未显示。



(a) 负数样例：输出位置 25461 ps



(b) 边界样例：输出位置 33010 ps

图 1-7 vadd 负数与边界用例结果波形

负数与边界样例的输出低 64 位分别为 F0F2F4F6F8FAFCFE 和 FEFE000000007F80，游标显示各自输入、输出与提交位置。

1.5.2 指令、事件与波形联合定位

波形、CSV 与反汇编提供互补的观察方式。反汇编给出静态地址与机器字，事件 CSV 按字段记录被提取的变化，FST 保留更完整的信号历史。局部截图适合快速呈现某一窗口，完整文件则适合围绕具体疑点展开查看。报告按这些用途组织图、表与附件。

一种直接的定位顺序是先从提交记录取得 PC，再在对应反汇编中找到指令，随后用事件坐标打开执行窗口。对 vadd，还可以核对机器字、目的 v3 和结果低 64 位；如果三者与软件用例一致，就可以进一步沿信号层级查看操作数和控制条件。这条路径把静态程序、软件输出和内部波形连接到同一条被观察指令。

阅读结果值时需要注意数据展示方向。日志可以按元素顺序输出十进制字节，波形窗口也可以把一组字节合并显示为十六进制。通过元素宽度和低位位置统一表示后，正数、负数和边界三个用例均可以按同样步骤阅读。数据解释与时间定位在这里是两项独立工作：先确认数值对应，再查看它出现在哪个观察点。

信号索引记录了提交与执行端的信号层级、位宽和标识，帮助读者解释 CSV 中的字段。检查某个字段时，可结合索引确定原信号；恢复对应完整波形后，再查看该坐标附近的上下文。报告通过精选窗口与附件索引兼顾主要过程的阅读和具体疑点的深入分析。

1.5.3 10 月 25 日含时钟的波形调试记录

10 月 25 日的独立 FST 模型使用同一组既有硬件生成物构建，保留真实时钟信号。最终提取覆盖两个 VialuFix、八个提交槽和 127 个信号，检查 33249 个完整上升沿，关联三次 FU 输入、三次 FU 输出与三次 DPIC 提交观测，共 9 条事件。

采样取每个局部上升沿前的稳定低相值，FST 原始时间与周期计数分别保存。DPIC 记录经过 Rob.scala 中 delay=3 的观察路径，其坐标表示提交接口的观测时刻。选通信号 FST 可在 GTKWave 中查看时钟、操作数、握手与事务标识。本节原波形图仍对应 10 月 9 日；10 月 25 日记录由独立波形与事件索引定位。

该次构建的向量 ALU 接口见固定源码中的VIAluFix 包装层，用于对照操作数、控制与结果的连接。

1.6 阶段结论与复现资料

1.6.1 阶段结论与材料入口

阶段一完成了配套环境部署、启动差异定位、共同 flash 配置和三类向量加法用例的差分运行。软件输出、EMU 退出状态、事件 CSV 和原波形为后续分析提供了互相对应的记录；自定义 vdot 的语义、父级控制与性能验证在后续阶段展开。

阶段一证据目录为 team/evidence/stage1-20261009，包含 hello-vadd-summary.log、工具版本、子模块锁、boot-flash 反汇编、vadd-commit-events.csv 及 vector-fix-alu-events.csv。程序源码位于 team/workloads/hello-vadd，本文源目录中的 evidence 保存用于报告展示的对应材料。

阶段一原记录 • 环境记录

1.6.2 源码版本与依赖恢复

本阶段使用的配套源码由以下完整提交号标识。xs-env 提供环境入口，香山及其子模块决定硬件生成输入；NEMU、nexus-am 和 DRAMsim3 分别参与参考执行、程序构建和存储系统仿真。对这些组件同时记录版本，可以避免仅固定顶层仓库、依赖却随下载时间变化的问题。

表 1-6 源码版本与依赖恢复

组件	已验证提交
xs-env	b0b506481e839a5df8a580e76f5aa246dc60fe7c
香山硬件基线	16ae9ddcda54fc9a2fddffad73174cf793ac7814
NEMU	a48a11f208ff81ad765591d83271d1b339fb3861

表 1-6 源码版本与依赖恢复（续）

组件	已验证提交
nexus-am	086a238062fdfb3281b745a537b361983d0154b5
DRAMsim3	1d2a9bf6da8e050d975ef2669c42faa93943f489
DiffTest	62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0
yunsuan	cadd3c2f43096e253d61296b33ac697be8354e29

在团队仓库执行 bash team/scripts/prepare-dependencies.sh。该入口检出固定 xs-env，初始化配套 NEMU、nexus-am、DRAMsim3 及其递归子模块，并锁定 NEMU 的额外依赖。团队仓库自身的香山子模块通过 git clone --recurse-submodules 获取。脚本在发现已有依赖版本不一致或已跟踪文件有修改时停止，保留现场供核对，避免把不同输入混入一次恢复过程。

NEMU 额外依赖的提交记录见锁定脚本。其中 SimPoint 按 NEMU 已记录的子模块版本递归初始化，其余下载仓库也以锁定提交为准。初始化完成后，应核对实际检出的提交，而不是只确认下载命令返回成功；完整源码锁与后续构建日志共同说明产物的来源。

1.6.3 完整工具版本

工具链包括运行构建系统的 JDK、硬件生成与转换工具、宿主编译器、目标程序编译器，以及波形阅读工具。各类工具处理的输入不同，记录版本时也应保留完整输出；例如目标 RISC-V GCC 决定程序指令，而宿主 Clang 或 GCC 处理的是仿真器的 C++ 代码。

表 1-7 完整工具版本

工具	版本
Oracle GraalVM JDK / javac	21.0.12
LLVM / Clang	19.1.7
Mill 项目版本	0.12.3
Scala / Chisel	2.13.15 / 6.6.0，由项目解析
firtool	1.62.1，由本次项目依赖解析
Verilator	5.052，提交 ea338be98e1e838d3518809ce8899f85a009963c
RISC-V GCC	11.4.0，riscv64-linux-gnu- 前缀
GNU binutils	2.38

表 1-7 完整工具版本（续）

工具	版本
宿主 GCC / G++	11.4.0
Make / CMake / Ninja	4.3 / 3.22.1 / 1.10.1
Python / GTKWave	3.10.12 / 3.3.104

完整输出见 tool-versions.txt。Ubuntu 软件包清单用于说明原环境的软件来源，新环境优先安装所需工具并核对关键版本。Mill 读取仓库.mill-version 选择项目版本，启动器的安装来源与项目实际运行版本应分别识别；Scala、Chisel 和 firtool 则按本项目解析结果核对。

1.6.4 跨环境复现条件

环境部署涉及 Windows 上的工作树管理和 WSL 中的硬件构建。选择执行位置时，首先考虑文件所在的文件系统：Git 的索引检查需要访问大量文件，Scala、Chisel 和 C++ 构建也会产生大量中间产物。把工具放在相应文件系统的原生环境执行，可以减少跨环境文件访问，同时使路径、权限和工具行为更容易对应。

表 1-8 跨环境复现条件

工作对象	执行位置	配套记录
Windows NTFS 工作树中的 Git 检查、提交与推送	Windows 原生 Git，使用 Windows 路径	分支、完整提交号、工作区差异及索引状态
Linux 或 WSL 原生目录中的 Git 操作	Linux 原生 Git	实际仓库根目录与提交身份
Chisel/Scala、硬件生成、RISC-V 编译、EMU/NEMU 和波形分析	Linux 或 WSL 原生文件系统	工具版本、构建参数、输入文件及输出目录
Windows 与 Linux 之间的源码、程序和证据交换	固定快照或独立检出，在接收端核验	完整提交号、未提交差异、依赖版本及关键文件 SHA-256

开始构建前，记录仓库根目录、提交号和实际输入；输出单独放置，避免与正在分析的旧记录混合。源码中存在未提交修改时，提交号应与差异快照共同描述输入。跨环境传输后，再核对用于运行的源码和二进制摘要，这样报告中的程序、运行日志和波形才能对应到同一次实验。

这一安排也受到后续工程实践的支持。2026-10-24 的 R4 调用入口发布检查中，一

次由 WSL 访问 Windows 工作树的检查在约 300 秒后超时。核对确认未产生提交、没有残留进程或索引锁后，Windows 原生 Git 完成了剩余检查与提交。过程见原始发布检查和Issue 记录。这是文件系统与工具选择的实际案例；两次执行条件不同，因此记录的是处理过程，没有据此计算 Git 操作的加速比例。

1.6.5 10 月 25 日验证的版本与产物

T04 运行绑定 T17 硬件生成来源 34bae6d4acceac939f958199dae53c14bc1f6183，两次运行共同 BIN 的 SHA-256 为 d81437b44473cef2b0c5777330e13fd076567d4fc1a4296be35d971e51a4dcad。生产 EMU、FST 模型、普通 NEMU、flash 及波形产物各自保留独立摘要。

本报告证据目录 evidence/stage1-current-20261025/保存以下七份原件，其固定公开版本为 2690c737：

- production-run/emulator-output.log：英文 hello、身份状态、用例结果和整程序统计。
- production-run/run.json：实际命令、输入前后摘要、日志摘要与退出状态。
- wave/wave.json：采样定义、信号清单、事件及选通信号产物摘要。
- acceptance.json：软件输入等价、原输入核验、最终波形与工具检查汇总。
- RESULT.md：两次运行、FST 构建与完整产物的复现索引。
- trace-run/emulator-output.log：独立 FST 模型的原始串口与终态。
- trace-run/run.json：FST 采集命令、运行输入与退出状态。

1.6.6 技术资料索引

环境部署详细方法。

1.7 原始操作日志

1.7.1 10 月 25 日生产模型原始输出

以下列出该次构建或运行的记录。

```
emu compiled at Oct 24 2026, 09:22:45
Using simulated 32768B flash
use /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin as flash bin
Using simulated 128MB RAM
The image is
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/hello-vadd-riscv64-xs.bin
```

```
The reference model is
/home/lizufan/riscv-lab/t17-clean-20261024-0854/support-01/nemu/reference/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin

[src/monitor/image_loader.c:250,load_img] Read 12 bytes from file
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin to 0x0xf0000000
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 56742ms
```

1.7.2 10 月 25 日 FST 模型原始输出

以下列出该次构建或运行的记录。

```
emu compiled at Oct 25 2026, 01:16:11
Using simulated 32768B flash
use /home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin as flash bin
Using simulated 128MB RAM
The image is
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/hello-vadd-riscv64-xs.bin
The reference model is
/home/lizufan/riscv-lab/t17-clean-20261024-0854/support-01/nemu/reference/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin

[src/monitor/image_loader.c:250,load_img] Read 12 bytes from file
/home/lizufan/riscv-lab/t04-current-20261025-0054/build-01/workload/build/boot-flash.bin to 0x0xf0000000
hello xiangshan, I am LiZufan001, IP address=172.27.184.54
identity_status=configured; IP address is build-host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
Core 0: HIT GOOD TRAP at pc = 0x800002a4
Core-0 instrCnt = 16621, cycleCnt = 43249, IPC = 0.384309
Seed=0 Guest cycle spent: 43253 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 145975ms
```

1.7.3 程序构建原始日志

以下列出该次构建或运行的记录。

```
+ /home/riscv/riscv-lab/stage1-build-workload.sh:11: date -Is
```

```
2026-10-09T20:55:26+08:00
+ /home/riscv/riscv-lab/stage1-build-workload.sh:12: cd /home/riscv/riscv-lab/workloads/hello-vadd
+ /home/riscv/riscv-lab/stage1-build-workload.sh:13: python3 -
+ /home/riscv/riscv-lab/stage1-build-workload.sh:20: make ARCH=riscv64-xs MARCH=rv64gcv -j1
# Building hello-vadd [riscv64-xs] with AM_HOME {/home/riscv/riscv-lab/xs-env/nexus-am}
+ CC main.c
# Building lib-am [riscv64-xs]
+ AS src/nemu/common/mainargs.S
+ AR -> build/am-riscv64-xs.a
# Building lib-klib [riscv64-xs]
# Creating binary image [riscv64-xs]
+ LD -> build/hello-vadd-riscv64-xs.elf
+ OBJCOPY -> build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:21: riscv64-linux-gnu-gcc -c -march=rv64gc -mabi=lp64d
boot-flash.S -o build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:22: riscv64-linux-gnu-objcopy -O binary -j .text
build/boot-flash.o build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:23: riscv64-linux-gnu-objdump -d build/boot-flash.o
+ /home/riscv/riscv-lab/stage1-build-workload.sh:24: cp build/boot-flash.bin
/home/riscv/riscv-lab/stage1-evidence/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:25: sha256sum build/boot-flash.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:26: cp build/hello-vadd-riscv64-xs.txt
/home/riscv/riscv-lab/stage1-evidence/hello-vadd-disassembly.txt
+ /home/riscv/riscv-lab/stage1-build-workload.sh:27: sha256sum build/hello-vadd-riscv64-xs.elf
build/hello-vadd-riscv64-xs.bin
+ /home/riscv/riscv-lab/stage1-build-workload.sh:28: echo WORKLOAD_BUILD_PASS
WORKLOAD_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-workload.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-workload.sh:1: printf 'exit_code=%s end=%s\n' 0
2026-10-09T20:55:26+08:00
```

1.7.4 历史 hello 与 vadd 运行摘要

以下列出该次构建或运行的记录。

```
The image is /home/riscv/riscv-lab/workloads/hello-vadd/build/hello-vadd-riscv64-xs.bin
The reference model is /home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
The first instruction of core 0 has commited. Difftest enabled.
[src/monitor/image_loader.c:205,load_img] Loading flash image:
/home/riscv/riscv-lab/workloads/hello-vadd/build/boot-flash.bin
hello xiangshan, 我是 riscv-vdot-team, IPaddress=10.198.93.97
identity_status=placeholder; IPaddress is WSL host IPv4
RVV VLEN=128 bits; SEW=8 LMUL=1 AVL=8
vadd case=0 output: 9 9 9 9 9 9 9 9 PASS
vadd case=1 output: 254 252 250 248 246 244 242 240 PASS
vadd case=2 output: 128 127 0 0 0 0 254 254 PASS
HELLO_VADD_PASS cases=3 lanes=24
[PERF ][time=              42965] SimTop.l_soc.l3cacheOpt.slices_0.mshrAlloc: mshr_lateCore 0: HIT GOOD
TRAP at pc = 0x8000029c
Core-0 instrCnt = 16557, cycleCnt = 42962, IPC = 0.385387
Host time spent: 300823ms
      Exit status: 0
+ /home/riscv/riscv-lab/stage1-run-hello-vadd.sh:21: echo HELLO_VADD_EMU_RUN_PASS
HELLO_VADD_EMU_RUN_PASS
```

1.7.5 NEMU 构建原始日志

以下列出该次构建或运行的记录。

```
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:9: date -Is
2026-10-09T20:24:27+08:00
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:10: cd /home/riscv/riscv-lab/xs-env/NEMU
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:11: git rev-parse HEAD
a48a11f208ff81ad765591d83271d1b339fb3861
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:12: make riscv64-xs-ref_defconfig
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/nanopb'...
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/LibCheckpoint'...
Submodule 'resource/nanopb' (https://github.com/nanopb/nanopb) registered for path 'resource/nanopb'
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/LibCheckpoint/resource/nanopb'...
Submodule path 'resource/nanopb': checked out '671672b4d7994a9b07a307ae654885c7202ae886'
Cloning into '/home/riscv/riscv-lab/xs-env/NEMU/resource/gcpt_restore'...
/home/riscv/riscv-lab/xs-env/NEMU/scripts/config.mk:20: Warning: .config does not exists!
/home/riscv/riscv-lab/xs-env/NEMU/scripts/config.mk:21: To build the project, first run 'make menuconfig'.
+ CC confdata.c
+ CC expr.c
+ CC preprocess.c
+ CC symbol.c
+ CC util.c
+ YACC build/parser.tab.h
+ LEX build/lexer.lex.c
+ CC build/lexer.lex.c
+ CC build/parser.tab.c
+ CC conf.c
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/tools/kconfig/build/conf
+ CC fixdep.c
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/tools/fixdep/build/fixdep
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:13: /usr/bin/time -v make -j1
Makefile:40: You have set CONFIG_FLASH_IMG_PATH option, but the specified path '""' does not exist!
Cloning into 'resource/softfloat/repo'...
+ CC src/nemu-main.c
+ CC src/device/device.c
+ CC src/device/alarm.c
+ CC src/device/intr.c
+ CC src/device/flash.c
+ CC src/cpu/tcache.c
+ CC src/cpu/cpu-exec.c
+ CC src/cpu/instr_stat.c
+ CC src/cpu/difftest/ame/amuctrl.c
+ CC src/cpu/difftest/ame/msync.c
+ CC src/cpu/difftest/dut.c
+ CC src/cpu/difftest/ref.c
+ CC src/monitor/ui.c
+ CC src/monitor/watchpoint.c
+ CC src/monitor/monitor.c
+ CC src/monitor/fdt_rng_seed.c
+ CC src/monitor/aligncheck.c
+ CC src/monitor/image_loader.c
+ CC src/utils/state.c
+ CC src/utils/iqueue.c
+ CC src/utils/br_log.c
+ CC src/utils/expr.c
+ CC src/utils/log.c
+ CC src/utils/filename.c
+ CC src/utils/dynamic-config.c
```

+ CC src/utils/timer.c
+ CC src/memory/host-tlb.c
+ CC src/memory/vaddr.c
+ CC src/memory/paddr.c
+ CC src/isa/riscv64/system/mmu.c
+ CC src/isa/riscv64/system/priv.c
+ CC src/isa/riscv64/system/trigger.c
+ CC src/isa/riscv64/system/intr.c
+ CC src/isa/riscv64/system/timer.c
+ CC src/isa/riscv64/system/aia.c
+ CC src/isa/riscv64/reg.c
+ CC src/isa/riscv64/ame/raw_fp4_memory.c
+ CC src/isa/riscv64/clint.c
+ CC src/isa/riscv64/difftest/dut.c
+ CC src/isa/riscv64/difftest/ref.c
+ CC src/isa/riscv64/init.c
+ CC src/isa/riscv64/logo.c
+ CC src/isa/riscv64/instr/priv/system.c
+ CC src/isa/riscv64/instr/fp.c
+ CC src/isa/riscv64/instr/rvk/aes_common.c
+ CC src/isa/riscv64/instr/rvk/crypto_impl.c
+ CC src/isa/riscv64/instr/ame/mcompute_impl.c
+ CC src/isa/riscv64/instr/ame/mldst_transpose_fast.c
+ CC src/isa/riscv64/instr/ame/mreg.c
+ CC src/isa/riscv64/instr/ame/mldst_impl.c
+ CC src/isa/riscv64/instr/ame/mldst_fast.c
+ CC src/isa/riscv64/instr/decode.c
+ CC src/isa/riscv64/instr/rva/amo.c
+ CC src/isa/riscv64/instr/rvv/vldst_impl.c
+ CC src/isa/riscv64/instr/rvv/vfexp2.c
+ CC src/isa/riscv64/instr/rvv/vcompute_impl.c
+ CC src/isa/riscv64/instr/rvv/vreg_impl.c
+ CC src/isa/riscv64/instr/rvv/vcommon.c
+ CC src/isa/riscv64/instr/rvcbo/cbo_impl.c
+ CC src/isa/riscv64/instr/mp.c
+ CC src/isa/riscv64/instr/rvb/rvintrin.c
+ CC src/isa/riscv64/instr/rvt/sfu.c
+ CC src/isa/riscv64/instr/rvt/matmul_subfloat.c
+ CC src/isa/riscv64/instr/rvt/element_wise.c
+ CC src/isa/riscv64/instr/rvt/convert.c
+ CC src/isa/riscv64/instr/rvt/conv2d.c
+ CC src/isa/riscv64/instr/rvt/type.c
+ CC src/isa/riscv64/instr/rvt/tensor_init.c
+ CC src/isa/riscv64/instr/rvt/tensor_copy.c
+ CC src/isa/riscv64/instr/rvt/pooling.c
+ CC src/isa/riscv64/instr/rvt/register.c
+ CC src/isa/riscv64/instr/rvt/matmul.c
+ CC src/isa/riscv64/instr/vp.c
+ CC src/engine/interpreter/hostcall.c
+ CC src/engine/interpreter/fp.c
+ CC src/engine/interpreter/init.c
+ CC src/device/io/mmio.c
+ CC src/device/io/port-io.c
+ CC src/device/io/map.c
+ CC src/profiling/profiling_control.c
+ CC src/checkpoint/cpt_env.c
+ CXX src/checkpoint/simpoint.cpp
+ CXX src/checkpoint/path_manager.cpp
+ CXX src/checkpoint/serializer.cpp

```
+ CXX src/checkpoint/semantic_point.cpp
+ CXX src/base/output.cpp
+ CXX src/iostream3/zfstream.cpp
+ CXX src/cpu/difftest/ame/msync_queue_wrapper.cpp
+ CXX src/cpu/difftest/ame/amu_ctrl_queue_wrapper.cpp
+ CXX src/memory/store_queue_wrapper.cpp
+ CXX src/memory/elfloader.cpp
SPECIALIZE_TYPE=RISCV SOFTFLOAT_OPTS="-DSOFTFLOAT_ROUND_ODD -DINLINE_LEVEL=5
-DSOFTFLOAT_FAST_DIV32TO16 -DSOFTFLOAT_FAST_DIV64TO32 -fPIC" make -s -C
/home/riscv/riscv-lab/xs-env/NEMU/resource/softfloat/repo/build/Linux-x86_64-GCC all
mkdir -p resource/softfloat/build
ln -sf /home/riscv/riscv-lab/xs-env/NEMU/resource/softfloat/repo/build/Linux-x86_64-GCC/softfloat.a
resource/softfloat/build/softfloat.a
+ g++ /home/riscv/riscv-lab/xs-env/NEMU/build/riscv64-nemu-interpreter-so
        Command being timed: "make -j1"
        User time (seconds): 22.97
        System time (seconds): 7.18
        Percent of CPU this job got: 101%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:29.76
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 445324
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 1324
        Minor (reclaiming a frame) page faults: 1488436
        Voluntary context switches: 7740
        Involuntary context switches: 491
        Swaps: 0
        File system inputs: 227800
        File system outputs: 96112
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:14: cp .config /home/riscv/riscv-lab/stage1-evidence/nemu.config
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:15: file build/riscv64-nemu-interpreter-so
build/riscv64-nemu-interpreter-so: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically
linked, BuildID[sha1]=4841e67452e47681eecaff72b0e3526c02cec25f, not stripped
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:16: sha256sum build/riscv64-nemu-interpreter-so
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:17: date -Is
2026-10-09T20:25:12+08:00
+ /home/riscv/riscv-lab/stage1-build-nemu.sh:18: echo NEMU_BUILD_PASS
NEMU_BUILD_PASS
```

1.7.6 EMU 构建原始日志

以下列出该次构建或运行的记录。

```
+ /home/riscv/riscv-lab/stage1-build-emu.sh:12: date -Is
2026-10-09T20:42:28+08:00
+ /home/riscv/riscv-lab/stage1-build-emu.sh:13: cd /home/riscv/riscv-lab/xs-env/XiangShan
+ /home/riscv/riscv-lab/stage1-build-emu.sh:14: git rev-parse HEAD
16ae9ddcda54fc9a2fddffad73174cf793ac7814
+ /home/riscv/riscv-lab/stage1-build-emu.sh:15: free -h
```

```
               total        used        free      shared  buff/cache   available
Mem:            19Gi       623Mi        16Gi       2.0Mi       2.3Gi        18Gi
Swap:           16Gi        73Mi        15Gi
+ /home/riscv/riscv-lab/stage1-build-emu.sh:16: /usr/bin/time -v make emu CONFIG=MinimalConfig
NUM_CORES=1 EMU_THREADS=1 EMU_TRACE=fst JVM_XMX=12G -j4
make -C ./difftest emu SIM_TOP=SimTop DESIGN_DIR=/home/riscv/riscv-lab/xs-env/XiangShan
NUM_CORES=1 RTL_SUFFIX=sv
make[1]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
Fri, 09 Oct 2026 20:42:28 +0800
Building emu...
make[2]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
time -avp -o /home/riscv/riscv-lab/xs-env/XiangShan/build/time.log make -s VM_PARALLEL_BUILDS=1
OPT_SLOW="-O0" \
                              OPT_FAST=-O3 \
                              PGO_CFLAGS= \
                              PGO_LDFLAGS= \
                              -C /home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile -f
                              /home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile/VSimTop.mk
make[3]: Entering directory '/home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile'
make[3]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/build/emu-compile'
make[2]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
make[1]: Leaving directory '/home/riscv/riscv-lab/xs-env/XiangShan/difftest'
        Command being timed: "make emu CONFIG=MinimalConfig NUM_CORES=1 EMU_THREADS=1
        EMU_TRACE=fst JVM_XMX=12G -j4"
        User time (seconds): 2451.21
        System time (seconds): 187.35
        Percent of CPU this job got: 397%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 11:03.88
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
        Average stack size (kbytes): 0
        Average total size (kbytes): 0
        Maximum resident set size (kbytes): 1301648
        Average resident set size (kbytes): 0
        Major (requiring I/O) page faults: 479
        Minor (reclaiming a frame) page faults: 50883606
        Voluntary context switches: 5000
        Involuntary context switches: 4568
        Swaps: 0
        File system inputs: 8832
        File system outputs: 3129248
        Socket messages sent: 0
        Socket messages received: 0
        Signals delivered: 0
        Page size (bytes): 4096
        Exit status: 0
+ /home/riscv/riscv-lab/stage1-build-emu.sh:17: ./build/emu --help
+ /home/riscv/riscv-lab/stage1-build-emu.sh:18: sha256sum build/emu
+ /home/riscv/riscv-lab/stage1-build-emu.sh:19: cp build/time.log
/home/riscv/riscv-lab/stage1-evidence/emu-time.log
+ /home/riscv/riscv-lab/stage1-build-emu.sh:20: date -Is
2026-10-09T20:53:32+08:00
+ /home/riscv/riscv-lab/stage1-build-emu.sh:21: echo XIANGSHAN_EMU_BUILD_PASS
XIANGSHAN_EMU_BUILD_PASS
+ /home/riscv/riscv-lab/stage1-build-emu.sh:1: rc=0
++ /home/riscv/riscv-lab/stage1-build-emu.sh:1: date -Is
+ /home/riscv/riscv-lab/stage1-build-emu.sh:1: printf 'exit_code=%s end=%s\n' 0 2026-10-09T20:53:32+08:00
```

1.7.7 工具与编译链版本原始输出

以下列出该次构建或运行的记录。

```
java 21.0.12 2026-07-21 LTS
Java(TM) SE Runtime Environment Oracle GraalVM 21.0.12+7.1 (build 21.0.12+7-LTS-jvmci-23.1-b96)
Java HotSpot(TM) 64-Bit Server VM Oracle GraalVM 21.0.12+7.1 (build 21.0.12+7-LTS-jvmci-23.1-b96,
mixed mode, sharing)
javac 21.0.12
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Ubuntu clang version 19.1.7 (++20250114103320+cd708029e0b2-1~exp1~20250114103432.75)
Target: x86_64-pc-linux-gnu
Thread model: posix
InstalledDir: /usr/lib/llvm-19/bin
Verilator 5.052 2026-09-05 rev v5.052
LLVM (http://llvm.org/):
  LLVM version 18.0.0git
  Optimized build.
CIRCT firtool-1.62.1
riscv64-linux-gnu-gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

GNU assembler (GNU Binutils for Ubuntu) 2.38
Copyright (C) 2022 Free Software Foundation, Inc.
This program is free software; you may redistribute it under the terms of
the GNU General Public License version 3 or later.
This program has absolutely no warranty.
This assembler was configured for a target of `riscv64-linux-gnu'.
Mill Build Tool version 0.12.3
Java version: 21.0.12, vendor: Oracle Corporation, runtime: /opt/graalvm-jdk-21
Default locale: en, platform encoding: UTF-8
OS name: "Linux", version: 6.6.87.2-microsoft-standard-WSL2, arch: amd64

[Scala/Chisel dependency versions]
val defaultScalaVersion = "2.13.15"
  "chisel"        -> ivy"org.chipsalliance::chisel:6.6.0",
  "chisel-plugin" -> ivy"org.chipsalliance:::chisel-plugin:6.6.0",
  "chiseltest"    -> ivy"edu.berkeley.cs::chiseltest:6.0.0"
  override def scalaVersion = defaultScalaVersion
  def scalaVersion: T[String] = T(defaultScalaVersion)
    def scalaVersion: T[String] = T(defaultScalaVersion)
    def scalaReflectIvy = ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}"
    def scalaVersion: T[String] = T(defaultScalaVersion)
    def scalaVersion: T[String] = T(defaultScalaVersion)
  override def scalaVersion: T[String] = T(defaultScalaVersion)
  override def ivyDeps = super.ivyDeps() ++ Agg(ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}")
  def scalaReflectIvy = ivy"org.scala-lang:scala-reflect:${defaultScalaVersion}"
```

1.7.8 固定子模块版本清单

以下列出该次构建或运行的记录。

1d2a9bf6da8e050d975ef2669c42faa93943f489 DRAMsim3 (heads/cosim-kmh)
68739c5a23cc48c9a13d28793834e4fee95d33ee DRAMsim3/ext/SuperLU_MT_3.1 (68739c5)
a48a11f208ff81ad765591d83271d1b339fb3861 NEMU (a48a11f)
e51a936d7fddfa03c81692039f184ab6c437e99e NEMU/resource/simpoint/simpoint_repo (heads/main)
bba8f3a4e3a15ef135baed37566be3e8a0c7d384 NutShell (bba8f3a)
29e80f8525735d1cb77f30fd2a3ea5eff3e6528c NutShell/difftest (29e80f8)
16ae9ddcda54fc9a2fddffad73174cf793ac7814 XiangShan (16ae9dd)
82079f4081bbc1f0221c790fbf8a29b1f23f48b8 XiangShan/coupledL2 (82079f4)
8faa17c9f7115782da1bd1665bd95587fba525ec XiangShan/coupledL2/HuanCun (8faa17c)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/coupledL2/HuanCun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/coupledL2/HuanCun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/HuanCun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/HuanCun/Utility/rocket-chip/torture/env (ce70afb)
90ae9a9314d444ade097fd5a471916869559b4fd XiangShan/coupledL2/HuanCun/rocket-chip (90ae9a9)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/HuanCun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/HuanCun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/HuanCun/rocket-chip/torture/env (ce70afb)
90ae9a9314d444ade097fd5a471916869559b4fd XiangShan/coupledL2/rocket-chip (90ae9a9)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/rocket-chip/torture/env (ce70afb)
992ce71166405de47c93102100a897726bd93bac XiangShan/coupledL2/utility (992ce71)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/coupledL2/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/coupledL2/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/coupledL2/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/coupledL2/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/coupledL2/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/coupledL2/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/coupledL2/utility/rocket-chip/torture/env (ce70afb)
62e4540c03c03aa947bdb67f4b6bf66f1a29c5d0 XiangShan/difftest (62e4540)
e1bd4695ca7beb36a5ce7357e9527ad9e95b9ec1 XiangShan/fudian (heads/pipeline)

b64af41c3276f97f0e181920400ee056b9c88037 XiangShan/fudian/berkeley-softfloat-3 (b64af41)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/fudian/berkeley-testfloat-3 (06b2007)
90aaf5935206ff322e461c3d021436c20dd0ac85 XiangShan/huancun (90aaf59)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/huancun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/huancun/Utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/huancun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/huancun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/huancun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/huancun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/huancun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/huancun/Utility/rocket-chip/torture/env (ce70afb)
e64b744b1490992bd038aaf9ca628f0e29fa0e59 XiangShan/huancun/rocket-chip (e64b744)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/huancun/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/huancun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/huancun/rocket-chip/hardfloat/berkeley-softfloat-3
(5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/huancun/rocket-chip/hardfloat/berkeley-testfloat-3
(06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/huancun/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/huancun/rocket-chip/torture/env (ce70afb)
614ceb4c5d39d32e6ae786693a35da429ea32c12 XiangShan/openLLC (614ceb4)
d3b04fef92092f9b27023bea57cc2ca99a8ffea0 XiangShan/openLLC/coupledL2 (d3b04fe)
90aaf5935206ff322e461c3d021436c20dd0ac85 XiangShan/openLLC/coupledL2/HuanCun (90aaf59)
de50694f6ffb48780a4b4665907442b3e4dd0d6d XiangShan/openLLC/coupledL2/HuanCun/Utility (de50694)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip
(175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d
XiangShan/openLLC/coupledL2/HuanCun/Utility/rocket-chip/torture/env (ce70afb)
e64b744b1490992bd038aaf9ca628f0e29fa0e59 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip
(e64b744)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/cde
(52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f
XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/torture
(99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d
XiangShan/openLLC/coupledL2/HuanCun/rocket-chip/torture/env (ce70afb)
bb4baf85c5bd4b55ffdcda12a75648fef212ab69 XiangShan/openLLC/coupledL2/rocket-chip (bb4baf8)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/rocket-chip/hardfloat
(d93aa57)

5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/coupledL2/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/coupledL2/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/rocket-chip/torture/env
(ce70afb)
342e0ad98bf24bf550f4d44099bd64875145d07c XiangShan/openLLC/coupledL2/utility (342e0ad)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/coupledL2/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/coupledL2/utility/rocket-chip/cde
(52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat
(d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/coupledL2/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/coupledL2/utility/rocket-chip/torture
(99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/coupledL2/utility/rocket-chip/torture/env
(ce70afb)
546a621ed82302db1bd1edc900b17ba55682c15c XiangShan/openLLC/openNCB (546a621)
c0955429aa3d9fe2be560f28599ece700fb0466c XiangShan/openLLC/openNCB/external/rocket-chip (c095542)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/openNCB/external/rocket-chip/cde
(52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat
(d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/openNCB/external/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/openNCB/external/rocket-chip/torture
(99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/openNCB/external/rocket-chip/torture/env
(ce70afb)
bb4baf85c5bd4b55ffdcda12a75648fef212ab69 XiangShan/openLLC/rocket-chip (bb4baf8)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/openLLC/rocket-chip/hardfloat/berkeley-softfloat-3
(5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/rocket-chip/torture/env (ce70afb)
342e0ad98bf24bf550f4d44099bd64875145d07c XiangShan/openLLC/utility (342e0ad)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/openLLC/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/openLLC/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/openLLC/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d
XiangShan/openLLC/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612
XiangShan/openLLC/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/openLLC/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/openLLC/utility/rocket-chip/torture/env (ce70afb)
ed8c30e1ec8a9c9aa6b116ce54281235d00273fa XiangShan/ready-to-run (ed8c30e)
bea0af5d855a871699b907c1988961141bbf28e7 XiangShan/rocket-chip (bea0af5)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)

06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/rocket-chip/torture/env (ce70afb)
9464aee3c77b021a037f28aa1b3fe53c71b516f6 XiangShan/src/main/resources/aia (heads/main)
a8ca0c6564865ee69b564d431ecbc52074ba8060 XiangShan/utility (a8ca0c6)
175dfe096e3b7c630f93ef328df1cf0b2ed55de1 XiangShan/utility/rocket-chip (175dfe0)
52768c97a27b254c0cc0ac9401feb55b29e18c28 XiangShan/utility/rocket-chip/cde (52768c9)
d93aa570806013dea479a92ba9bb33d1f2d4f69f XiangShan/utility/rocket-chip/hardfloat (d93aa57)
5c06db33fc1e2130f67c045327b0ec949032df1d XiangShan/utility/rocket-chip/hardfloat/berkeley-softfloat-3 (5c06db3)
06b20075dd3c1a5d0dd007a93643282832221612 XiangShan/utility/rocket-chip/hardfloat/berkeley-testfloat-3 (06b2007)
99d8b2b441ecaa18b852505bb7718ee04e2753f5 XiangShan/utility/rocket-chip/torture (99d8b2b)
ce70afbf50a203be04bc326326cfa75831fe7f5d XiangShan/utility/rocket-chip/torture/env (ce70afb)
cadd3c2f43096e253d61296b33ac697be8354e29 XiangShan/yunsuan (cadd3c2)
086a238062fdfb3281b745a537b361983d0154b5 nexus-am (086a238)

1.7.9 原始波形事件记录

以下列出该次构建或运行的记录。

cycle,scope,pc,instruction,vecwen,wdest
19562,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0xcc0472d7,0,5
19624,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x2050087,1,1
19624,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000134,0x2058107,1,2
19631,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x21101d7,1,3
19646,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x20601a7,1,32
19646,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,0x28513,0,10
19647,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000142,0x8067,0,0
25442,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0xcc0472d7,0,5
25458,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x2050087,1,1
25459,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000134,0x2058107,1,2
25466,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x21101d7,1,3
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x20601a7,1,32
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,0x28513,0,10
25481,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_2.dpic,0x80000142,0x8067,0,0
32991,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000012c,0xcc0472d7,0,5
33007,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000130,0x2050087,1,1
33008,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000134,0x2058107,1,2

33015,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x80000138,0x
21101d7,1,3
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module.dpic,0x8000013c,0x
20601a7,1,32
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_1.dpic,0x80000140,
0x28513,0,10
33030,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_ctrlBlock.rob.difftest_module_2.dpic,0x80000142,
0x8067,0,0

1.7.10 向量运算单元事件记录

以下列出该次构建或运行的记录。

cycle,scope,phase,fuOpType,vsew,src0,src1,result
19625,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
fff0102030405060708,0xffffffffffffffff0807060504030201,
19626,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
f0909090909090909
25460,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
ffff8f9fafbfcfdfeff,0xffffffffffffffff8f9fafbfcfdfeff,
25461,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
ff0f2f4f6f8fafcfe
33009,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,in,64,0,0xffffffffffff
fffff7f80ff0001ff01,0xffffffffffffffff7f800100ff807f,
33010,TOP.SimTop.l_soc.core_with_l2.core.backend.inner_vfExuBlock.exus_2.VialuFix,out,,,,,0xffffffffffffff
ffefe000000007f80